

Practical No.:	07
Practical Title :	To install and Configure Pull based Software Configuration Management and provisioning tools using Puppet /Ansible.
Aim:	To install and configure pull-based configuration management tools like Puppet or Ansible.
Objective:	To understand the purpose of configuration management tools in automating system administration. To install and configure Puppet or Ansible on a Linux system.

Theory:**What is a configuration management tool?**

A major part of the DevOps process is configuration management. Here, IT admins use a tool, or set of tools, to model existing infrastructure and create configurations to provision onto that infrastructure. These tools enable admins to monitor configurations as they run and remediate any problems that occur within the runtime environment.

The process progresses like so:

1. Build a blueprint of infrastructure that's already there.
2. Define the desired end result.
3. Create the means to achieve this result.
4. Carry out the changes.
5. Ensure that the desired result is achieved.
6. Monitor the system.
7. Make any required changes.

What is Puppet?

There are various tools on the market, both open source and commercial systems that offer these configuration management capabilities. One tool with a strong following is Puppet, which is available as an open source tool, as well as a fully supported commercial version, Puppet Enterprise.

Understanding Puppet architecture

Although Puppet can run in a server-only model with command-line access, most users run it in full client-server mode, where the servers run as masters and the clients run as agents. For high availability, you should use more than one server. The Puppet agent requests and receives individual catalogs from the server and then enforces that state on the node for which it is responsible. Each agent then maintains reports that are sent back to the server. The server monitors and manages the overall platform; it also provides the data and configuration files that the agents request and receives and manages data coming back from the agents.

Puppet's architecture relies on several key components of the primary server environment:

1. Puppet master. The application that compiles and stores Puppet code and data. Its main purpose is to process the configuration manifests to inventory information, which it then distributes to agents.
2. Puppet agent. The application that runs on every managed node. The role of the agent is to send facts to the master to locally apply the latest catalog.
3. Facts. Variables that make up a node, such as OSes and IP addresses. Puppet compiles facts

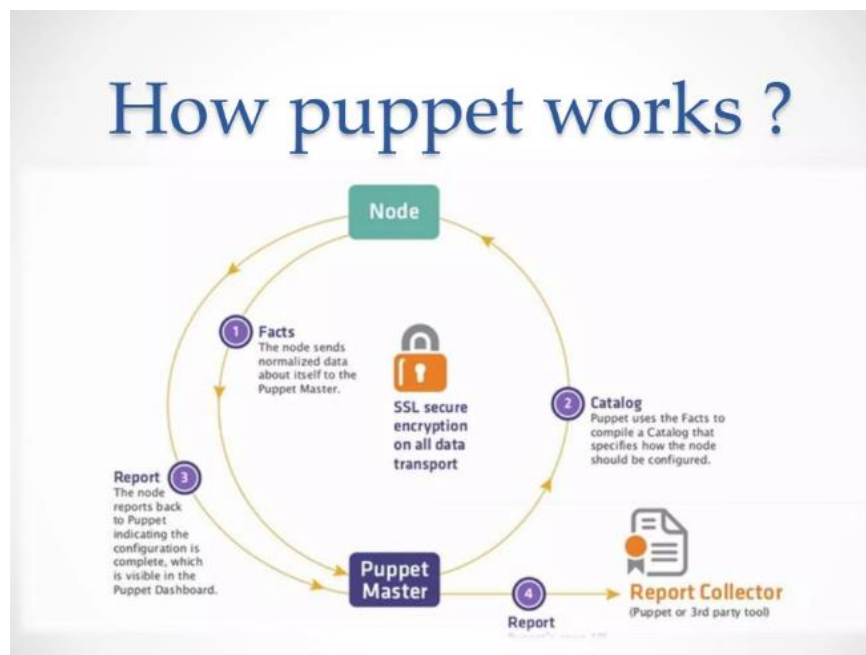
for the right catalog for each node.

4. Manifests. Composed of Puppet code and tells Puppet how to configure a system. They are written in Puppet's native language and saved with the .pp extension.

5. Templates. Final documents created from the combination of code and data.

6. Files. Static content that can be downloaded.

7. Modules. A single directory structure made up of manifests and data, such as facts, templates and files.



Puppet Terminologies

Facts

Facter is a cross platform Ruby library designed to gather information on puppet nodes.

```
[root@puppetnode ~]# facter -y
---
macaddress_eth0: "08:00:27:62:B9:AD"
blockdevice_sda_vendor: ATA
operatingsystem: CentOS
rubyversion: "1.8.7"
is_virtual: "true"
productname: VirtualBox
swapfree: "3.94 GB"
hardwareisa: x86_64
id: root
kernelmajversion: "2.6"
interfaces: "eth0,eth1,lo"
swapspace_mb: "4031.99"
uuid: "78347BE7-62D1-4ECF-BD30-2159CF8FDA3C"
netmask: "255.255.255.0"
swapspace: "3.94 GB"
bios_release_date: "12/01/2006"
processor0: "Intel(R) Core(TM) i7-3520M CPU @ 2.90GHz"
uniqueid: "1cac0580"
```

Puppet Terminologies

Resources

```
type { 'title':
  attribute => value,
}
file { '/etc/passwd':
  ensure => file,
  owner  => 'root',
  group  => 'root',
  mode   => '0600',
}
puppet resource user root
```

Classes

Group of resources can be organised in classes.

Nodes

```
Classes get applied to nodes.
# /etc/puppet/manifests/site.pp
node 'test.corp.adobe.com' {
  include common
  include apache
  include squid
}
```

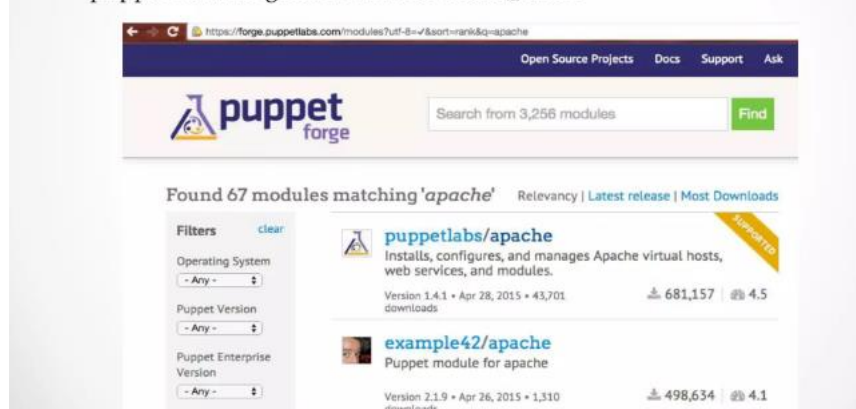
Relationships and Ordering

- Order of resources in Puppet does not matter. Puppet manages resources in most efficient way
- If some resources are to be managed in specific order, declare relationships :

```
package { 'openssh-server':  
  ensure => exists,  
  before => File['/etc/ssh/sshd_config']  
}  
OR  
file { '/etc/ssh/sshd_config':  
  ensure => file,  
  mode   => 0600,  
  source => 'puppet:///modules/sshd/sshd_config',  
  require => Package['openssh-server']  
}  
notify, subscribe, chaining arrows (-> and ~>)
```

Puppet Modules

puppet module install puppetlabs-apache
puppet module generate author-mode_name



Puppet Installation and Configuration

Step 1: Install Puppet Server and Agent

On the Puppet Master (Server)

1. Install Puppet Server

sudo apt update

sudo apt install puppetserver -y

2. Start and Enable Puppet Server

sudo systemctl start puppetserver

sudo systemctl enable puppetserver

3. Configure Puppet Server (Modify puppet.conf if needed)

sudo nano /etc/puppetlabs/puppet/puppet.conf

Ensure the following settings are present:

[main]

server = puppetmaster.example.com

environment = production

4. Restart Puppet Server

sudo systemctl restart puppetserver

On the Puppet Agent (Client Node)

1. Install Puppet Agent

sudo apt update

sudo apt install puppet-agent -y

2. Start and Enable Puppet Agent

sudo systemctl start puppet

sudo systemctl enable puppet

3. Configure Puppet Agent

sudo nano /etc/puppetlabs/puppet/puppet.conf

Add:

[main]

server = puppetmaster.example.com

4. Sign the Certificate on the Master Node

sudo puppetserver ca list

sudo puppetserver ca sign -all

5. Run Puppet Agent

sudo puppet agent -test

Ansible Installation and Configuration

Step 1: Install Ansible

1. On the Ansible Control Node (Master)

sudo apt update

sudo apt install ansible -y

2. Verify Installation

ansible --version

Step 2: Configure Ansible Inventory and SSH Access

1. Edit the Hosts File

sudo nano /etc/ansible/hosts

Add managed nodes:

[web]

node1.example.com

node2.example.com

2. Set Up SSH Key Authentication

```
ssh-keygen -t rsa
```

```
ssh-copy-id user@node1.example.com
```

```
ssh-copy-id user@node2.example.com
```

3. Test Connectivity

```
ansible all -m ping
```

Step 3: Create and Apply a Playbook**1. Create an Ansible Playbook**

```
nano install_apache.yml
```

Add the following content: - hosts: web

become: yes

tasks: - name: Install Apache

apt:

name: apache2

state: present

2. Run the Playbook

```
ansible-playbook install_apache.yml
```

Conclusion:
